

PROBLEM SETS

PROBLEM SET 01: Representing and Manipulating Information

Exercise in the textbook **Computer Systems: A Programmer's Perspective**: 2.44, 2.45, 2.47, 2.50, 2.52, 2.54 (Practice Problems)...

Problem 1: Multiple choice questions

1. Which of the following is a legitimate difference between IA-32 and x86-64?
 - (a) Buffer overflow exploits are impossible under x86-64.
 - (b) IA-32 has caller- and callee-saved register conventions, while x86-64 does not.
 - (c) Under x86-64, any instructions that take 32-bit operands are illegal.
 - (d) None of the above.
2. Imagine a floating point format with no sign bit, one exponent bit, and one fraction bit. Which of the following is not a number?
 - (a) 00
 - (b) 01
 - (c) 10
 - (d) 11
 - (e) None of the above
3. In IEEE floating point, what would be an effect of allocating more bits to the exponent part by taking them from the fraction part?
 - (a) You could represent fewer numbers, but they could be much larger.
 - (b) You could represent the same numbers, but with more decimal places.
 - (c) You could represent both larger and smaller numbers, but with less precision.
 - (d) Some previously representable numbers would now round to infinity.
4. What is the minimum (most negative) value of a 32-bit two's complement integer?
 1. -2^{32}
 2. $-2^{32} + 1$
 3. -2^{31}
 4. $-2^{31} + 1$

-
5. After executing the following code, which of the variables are equal to 0?

```
unsigned int a = 0xffffffff;
unsigned int b = 1;
unsigned int c = a + b;
unsigned long d = a + b;
unsigned long e = (unsigned long)a + b;
```

(Assume ints are 32 bits wide and longs are 64 bits wide.)

1. None of them
 2. c
 3. c and d
 4. c, d, and e
6. What is the output of the following code?
- Assume that `int` is 32 bits, `short` is 16 bits, and the representation is two's complement.

```
unsigned int x = 0xDEADBEEF;
unsigned short y = 0xFFFF;
signed int z = -1;
```

```
if (x > (signed short) y)
    printf("Hello");
if (x > z)
    printf("World");
```

- (a) Prints nothing.
 - (b) Prints "Hello"
 - (c) Prints "World"
 - (d) Prints "HelloWorld"
7. Which expression will evaluate to 0x1 if `x` is a multiple of 32 and 0x0 otherwise?
- Assume that `x` is an `unsigned int`.
- (a) `!(x & 0x1f)`
 - (b) `!(x & 0x3f)`
 - (c) `(x & 0x1f)`
 - (d) `(x | 0x3f)`
 - (e) `!(x ^ 0x1f)`
8. On a 32-bit Linux system, what is the size of a `long`?
- (a) 2 bytes
 - (b) 4 bytes
 - (c) 6 bytes
 - (d) 8 bytes
 - (e) 16 bytes

9. Which of the following 8-bit floating point numbers (1 sign, 3 exponent, 4 fraction) represent NaN?

- (a) 1 000 1111
- (b) 0 111 1111
- (c) 0 100 0000
- (d) 1 111 0000

10. Consider the following code, what is the output of the `printf`?

```
int x = 0x15213F10 >> 4;
char y = (char) x;
unsigned char z = (unsigned char) x;
printf("%d, %u", y, z);
```

- (a) -241, 15
- (b) -15, 241
- (c) -241, 241
- (d) -15, 15

11. In two's complement, what is $-TMin$?

- (a) $TMin$
- (b) $TMax$
- (c) 0
- (d) -1

12. In C, the expression `15213U > -1` evaluates to:

- (a) True (1)
- (b) False (0)

13. Let `int x = -31 / 8;` and `int y = -31 >> 3;`. What are the values of `x` and `y`?

- (a) `x = -3, y = -3`
- (b) `x = -4, y = -4`
- (c) `x = -3, y = -4`
- (d) `x = -4, y = -3`

14. In two's complement, what is the minimum number of bits needed to represent the numbers -1 and 1 respectively?

- (a) 1 and 2
- (b) 2 and 2
- (c) 2 and 1
- (d) 1 and 1

-
15. Consider the following program. Assuming the user correctly types an integer into stdin, what will the program output?

```
#include <stdio.h>
int main(){
    int x = 0;
    printf("Please input an integer:");
    scanf("%d", x);
    printf("%d", (!!x)<<31);
}
```

- (a) 0
- (b) TMin
- (c) Depends on the integer read from stdin
- (d) Segmentation fault

Problem 2:

Consider the following 7-bit floating point representation based on the IEEE floating point format:

- There is a sign bit in the most significant bit.
- The next $k = 3$ bits are the exponent. The exponent bias is 3.
- The last $n = 3$ bits are the fractional part.

Numeric values are encoded in this format as a value of the form

$$V = (-1)^s \times M \times 2^E,$$

where s is the sign bit, E is exponent after biasing, and M is the significand.

Part I

Answer the following problems using either decimal (e.g., 1.375) or fractional (e.g., 11/8) representations for numbers that are not integers.

A. For denormalized numbers:

- (a) What is the value E of the exponent after biasing? _____
- (b) What is the largest value M of the significand? _____

B. For normalized numbers:

- (a) What is the smallest value E of the exponent after biasing? _____
- (b) What is the largest value E of the exponent after biasing? _____
- (c) What is the largest value M of the significand? _____

Part II

Fill in the blank entries in the following table giving the encodings for some interesting numbers.

Description	E	M	V	Binary Encoding
Zero		0	0	0 000 0000
Smallest Positive (nonzero)				
Largest denormalized				
Smallest positive normalized				
One			1	
Largest finite number				
NaN	—	—	NaN	
Infinity	—	—	$+\infty$	

Problem 3:

In this problem, you will work with floating-point numbers based on the IEEE floating-point format. We consider two different 6-bit formats:

Format A:

- There is one sign bit s .
- There are $k = 3$ exponent bits. The bias is $2^{k-1} - 1 = 3$.
- There are $n = 2$ fraction bits.

Format B:

- There is one sign bit s .
- There are $k = 2$ exponent bits. The bias is $2^{k-1} - 1 = 1$.
- There are $n = 3$ fraction bits.

For formats A and B, please write down the binary representation for the following (use round-to-even). Recall that for denormalized numbers, $E = 1 - \text{bias}$. For normalized numbers, $E = e - \text{bias}$.

Value	Format A Bits	Format B Bits
One	0 011 00	0 01 000
Three		
$7/8$		
$15/8$		

Problem 4:

Use the following 8-bit floating point representation: 1 sign bit, 3 exponent bits, and 4 fraction bits.

A. What is the bias? _____

B. What is the smallest positive value that can be represented? _____

-
- C. What is the largest positive denormalized number? _____
- D. What is the representation of negative infinity? _____
- E. Fill in the following table. Use round-to-even. Value should be written to decimal

Bits	Decimal Value
0 000 0000	0
1 010 0000	
	−13
	$\frac{1}{16}$
0 111 1111	
	$\frac{15}{256}$

Problem 5:

In this problem, you will work with a simplified 5-bit floating-point representation based on the IEEE floating-point format. This format does not have a sign bit – it can only represent nonnegative numbers.

Floating-point Format Specification:

- There are $k = 3$ exponent bits. The exponent bias is 3.
- There are $n = 2$ fraction bits.

Numeric values are encoded as a value of the form $V = M \times 2^E$, where E is the exponent after biasing, and M is the significand. The fraction bits encode the significand M using either a denormalized representation (when the exponent field is 0) or a normalized representation (when the exponent field is nonzero). Any rounding of the significand should use the *round-to-even* rule. You are given several decimal values. Your task is to encode them using the 5-bit floating point format, and also write down the rounded (decoded) value corresponding to each encoding. Please express rounded values as reduced fractions (e.g., $3/4$) or integers (e.g., 17).

Value	Floating Point Bits	Rounded Value
9/32	001 00	1/4
1/32		
1/16		
3/32		
1		
11		
12		

Problem 6:

Integer encoding. Fill in the blanks in the table below with the number described in the first column of each row. You can give your answers as unexpanded simple arithmetic expressions (such as $15^{2^{13}} + 42$); you should not have trouble fitting your answers into the space provided.

For this problem, assume a **6-bit word size**.

Description	Number
U_{max}	
T_{min}	
(unsigned) ((int) 4)	
(unsigned) ((int) -7)	
((unsigned) 0x21) << 1 & 0x3F	
(int) (20 + 12)	
12 && 4	
(! 0x15) > 16	

PROBLEM SET 02: Machine-Level Representation of Programs

Exercise in the textbook **Computer Systems: A Programmer's Perspective**: 3.1, 3.3, 3.5, 3.10, 3.18, 3.21, 3.23, 3.24, 3.25 (Practice Problems)...

Problem 1: Multiple choice questions

- Which of the following is the correct ordering (left-to-right) of a file's compilation cycle (a filename with no extension is an executable):

- (a) `foo.c` $\rightarrow$ `foo.o` $\rightarrow$ `foo.s` $\rightarrow$ `foo`
- (b) `foo` $\rightarrow$ `foo.s` $\rightarrow$ `foo.o` $\rightarrow$ `foo.c`
- (c) `foo.c` $\rightarrow$ `foo.s` $\rightarrow$ `foo` $\rightarrow$ `foo.o`
- (d) `foo.c` $\rightarrow$ `foo.s` $\rightarrow$ `foo.o` $\rightarrow$ `foo`

- Consider the following two blocks of code, which are contained in *separate files*:

```
/* main.c */
int i = 0;
int main() {
    foo();
    return 0;
}

/* foo.c */
int i = 1;
void foo() {
    printf("%d", i);
}
```

What will happen when you attempt to compile, link, and run this code?

- (a) It will fail to compile.
 - (b) It will fail to link.
 - (c) It will raise a segmentation fault.
 - (d) It will print "0".
 - (e) It will print "1".
 - (f) It will sometimes print "0" and sometimes print "1".
- What section of memory holds the assembly for `printf`?
 - (a) Stack
 - (b) Kernel memory
 - (c) Shared libraries

-
- (d) Heap
6. Which of the following tools would you first use to debug an application which is exiting with the error “Segmentation fault”?
- (a) `gdb`
 - (b) `strace`
 - (c) `strings`
 - (d) `objdump`
7. How does x86 assembly store the return value when a function is finished?
- (a) The `ret` instruction stores it in a special `retval` register.
 - (b) By convention, it is always in `%eax`.
 - (c) It is stored on the stack just above the (`%ebp`) of the callee.
 - (d) It is stored on the stack just above all the arguments to the function.
8. Which of the following is FALSE concerning x86-64 architecture?
- (a) A `double` is 64 bits long.
 - (b) Registers are 64 bits long.
 - (c) Pointers are 64 bits long.
 - (d) Pointers point to locations in memory that are multiples of 64 bits apart.
9. Which of the following x86 instructions can be used to add two registers and store the result without overwriting either of the original registers?
- (a) `mov`
 - (b) `lea`
 - (c) `add`
 - (d) None of the above
10. The register `%rax` is currently storing a NULL pointer. Which of the following x86 instructions will cause a segmentation fault because of an invalid memory access?
- (a) `mov (%rax), %rcx`
 - (b) `lea (%rax), %rcx`
 - (c) None of the above
11. What is the difference between the `mov` and `lea` instructions?
- 1. `lea` dereferences an address, while `mov` doesn't.
 - 2. `mov` dereferences an address, while `lea` doesn't.
 - 3. `lea` can be used to copy a register into another register, while `mov` cannot.
 - 4. `mov` can be used to copy a register into another register, while `lea` cannot.

-
12. Assume a function `foo` takes two arguments. When calling `foo(arg1, arg2)`, which is the correct order of operations assuming x86 calling conventions and that `foo` must allocate stack space (implies that we must save the `%ebp`)?
1. `push arg1, push arg2, call foo, push %ebp`
 2. `push arg1, push arg2, push %ebp, call foo`
 3. `push arg2, push arg1, call foo, push %ebp`
 4. `push arg2, push arg1, push %ebp, call foo`
13. Which one of the following statements is NOT true?
- (a) x86-64 provides a larger virtual address space than x86.
 - (b) The stack disciplines for x86 and x86-64 are different.
 - (c) x86 uses `%ebp` as the base pointer for the stack frame.
 - (d) x86-64 uses `%rbp` as the base pointer for the stack frame.
14. Which of the following accomplishes: `%eax = 5 * %eax`?
1. `mov (%eax, %eax, 4), %eax`
 2. `lea (%eax, %eax, 4), %eax`
- (a) Neither 1 nor 2.
 - (b) Only 1.
 - (c) Only 2.
 - (d) Both 1 and 2.
15. The x86-64 instruction `test` is best described as:
- (a) Same as `sub`.
 - (b) Same as `sub`, but doesn't keep the result (only sets flags).
 - (c) Same as `and`.
 - (d) Same as `and`, but doesn't keep the result (only sets flags).
16. Consider the C declaration:
- ```
int array[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```
- Suppose that the compiler has placed the variable `array` in the `%ecx` register. How do you move the value at `array[3]` into the `%eax` register? Assume that `%ebx` is 3.
- (a) `leal 12(%ecx), %eax`
  - (b) `leal (%ecx, %ebx, 4), %eax`
  - (c) `movl (%ecx, %ebx, 4), %eax`
  - (d) `movl 8(%ecx, %ebx, 2), %eax`
  - (e) `leal 4(%ecx, %ebx, 1), %eax`
17. What is **NOT** true about 64-bit Linux systems?
- (a) `%rax` is used for function return values.
  - (b) There are more registers than there are in 32-bit systems.
  - (c) All function arguments are passed on the stack.
  - (d) `%rbp` can be used like any other register; there is no base pointer.
  - (e) `%eax` and `%ebx` can be used like in a 32-bit system.

- 
18. Consider an `int *a` and an `int n`. If the value of `%ecx` is `a` and the value of `%edx` is `n`, which of the following assembly snippets best corresponds to the C statement `return a[n]`?
- (a) `ret (%ecx,%edx,4)`
  - (b) `leal (%ecx,%edx,4),%eax ret`
  - (c) `mov (%ecx,%edx,4),%eax ret`
  - (d) `mov (%ecx,%edx,1),%eax ret`
19. `%rsp` is `0xdeadbeefdeadd0d0`. What is the value in `%rsp` after the following instruction executes?
- `pushq %rbx`
- (a) `0xdeadbeefdeadd0d4`
  - (b) `0xdeadbeefdeadd0d8`
  - (c) `0xdeadbeefdeadd0cc`
  - (d) `0xdeadbeefdeadd0c8`
20. Which register holds the first argument when a function is called in IA32 (32-bit) architecture?
- (a) `edi`
  - (b) `esi`
  - (c) `eax`
  - (d) None of the above
21. What is the C equivalent of `mov 0x10(%rax,%rcx,4),%rdx`?
- (a) `rdx = rax + rcx + 4 + 10`
  - (b) `*(rax + rcx + 4 + 10) = rdx`
  - (c) `rdx = *(rax + rcx*4 + 0x10)`
  - (d) `rdx = *(rax + rcx + 4 + 0x10)`
22. What is the C equivalent of `leal 0x10(%rax,%rcx,4),%rdx`?
- (a) `rdx = 10 + rax + rcx + 4`
  - (b) `rdx = 0x10 + rax + rcx*4`
  - (c) `rdx = *(0x10 + rax + rcx*4)`
  - (d) `*(0x10 + rax + rcx + 4) = rdx`

---

## Problem 2: Assembly Programming

Consider the following pairs of C functions and assembly code. Fill in the missing instructions in the assembly code (one instruction per a blank). Your answers should be syntactically correct assembly.

|                                                                                                                    |                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int goose() {<br/>    return -4;<br/>}</pre>                                                                  | <pre>goose:<br/>    pushl    %ebp<br/>    movl     %esp, %ebp<br/>    -----<br/>    popl     %ebp<br/>    ret</pre>                                       |
| <pre>int cow(int a, int b) {<br/>    return a - b;<br/>}</pre>                                                     | <pre>cow:<br/>    pushl    %ebp<br/>    movl     %esp, %ebp<br/>    movl     8(%ebp), %eax<br/>    -----<br/>    popl     %ebp<br/>    ret</pre>          |
| <pre>int pig(int a) {<br/>    return a * 3;<br/>}</pre>                                                            | <pre>pig:<br/>    pushl    %ebp<br/>    movl     %esp, %ebp<br/>    movl     8(%ebp), %eax<br/>    leal     -----<br/>    popl     %ebp<br/>    ret</pre> |
| <pre>int sheep(int c) {<br/>    if (c &lt; 0)<br/>        return 1;<br/>    else<br/>        return 0;<br/>}</pre> | <pre>sheep:<br/>    pushl    %ebp<br/>    movl     %esp, %ebp<br/>    movl     8(%ebp), %eax<br/>    -----<br/>    popl     %ebp<br/>    ret</pre>        |

---

```

 duck:
 pushl %ebp
 movl %esp, %ebp
 pushl %ebx
 movl 8(%ebp), %ebx

 call sheep
 movl %ebx, %edx

 je .L6
 negl %edx
.L6:
 movl %edx, %eax
 addl $4, %esp
 popl %ebx
 popl %ebp
 ret
int duck(int a) {
 if (sheep(a))
 return -a;
 else
 return a;
}

```

### Problem 3: Assembly Programming

This problem tests your understanding of IA32 condition codes.

#### A.

Consider the instruction:

```
cmpl a, b
```

Write in the values (0 if clear, 1 if set) of the condition flags if this instruction is executed with the given values of *a* and *b*.

| a          | b           | Zero Flag (ZF) | Sign Flag (SF) | Carry Flag (CF) | Overflow Flag (OF) |
|------------|-------------|----------------|----------------|-----------------|--------------------|
| -4         | 0xffffffffc |                |                |                 |                    |
| 4          | 0xffffffffc |                |                |                 |                    |
| -1         | 1           |                |                |                 |                    |
| 2          | 0x80000000  |                |                |                 |                    |
| 0x7fffffff | 0x80000000  |                |                |                 |                    |
| 0x80000000 | 0x7fffffff  |                |                |                 |                    |
| 1          | 0x7fffffff  |                |                |                 |                    |
| 0x80000000 | 0x80000000  |                |                |                 |                    |
| 0x7fffffff | 0xffffffff  |                |                |                 |                    |

#### B.

On an IA32 architecture, compare and test instructions aren't the only instructions which set the condition codes and conditional branches aren't the only instructions which read the condition

---

codes. Specifically, the `add` instruction sets the condition codes based on the result and the `add with carry` instruction (`adc`) computes the sum of its two operands and the carry flag. That is,

```
adcl %edx, %eax
```

#### Problem 4:

Consider the following C functions and assembly code:

```
int fun1(int i, int j)
{
 if (i+3 != j)
 return i+3;
 else
 return j*16;
}
```

```
int fun2(int i, int j)
{
 if (i+3 != (unsigned)j)
 return i;
 else
 return j*4;
}
```

```
int fun3(int i, int j)
{
 if (i+3 <= (unsigned)j)
 return i;
 else
 return j >> 2;
}
```

```
pushl %ebp
movl %esp, %ebp
movl 8(%ebp), %eax
movl 12(%ebp), %ecx
leal 3(%eax), %edx
cmpl %ecx, %edx
jne .L4
leal 0(,%ecx,4), %eax
.L4:
popl %ebp
ret
```

Which of the functions compiled into the assembly code shown?

#### Problem 5:

This problem tests your understanding of how `for` loops in C relate to IA32 machine code. Consider the following IA32 assembly code for a procedure `dog()`:

```
dog:
 pushl %ebp
 movl %esp, %ebp
 movl 12(%ebp), %ecx
 movl $1, %eax
```

---

```

 movl 8(%ebp), %edx
 cmpl %ecx, %edx
 jge .L7
.L5:
 imull %edx, %eax
 addl $2, %edx
 cmpl %ecx, %edx
 jl .L5
.L7:
 popl %ebp
 ret

```

Based on the assembly code, fill in the blanks below in its corresponding C source code. (Note: you may only use symbolic variables `x`, `y`, `i`, and `result` from the source code in your expressions below — **do not** use register names.)

```

int dog(int x, int y)
{
 int i, result;

 result = _____;

 for (i = _____; _____; _____) {
 result = _____;
 }

 return result;
}

```

### Problem 6:

This problem tests your understanding of how `switch` statements in C relate to IA32 machine code. Consider the following IA32 assembly code for a procedure `frog()`:

```

frog:
 pushl %ebp
 movl %esp, %ebp
 movl 8(%ebp), %edx
 movl 12(%ebp), %eax
 cmpl $7, %edx
 ja .L8
 jmp *.L9(,%edx,4)

.section .rodata
.align 4
.align 4

```

---

```

.L9:
 .long .L8
 .long .L4
 .long .L8
 .long .L5
 .long .L8
 .long .L4
 .long .L6
 .long .L2
.text

.L4:
 movl $7, %eax
 jmp .L2

.L5:
 decl %eax
 jmp .L2

.L6:
 incl %eax
 jmp .L2

.L8:
 movl $-1, %eax

.L2:
 popl %ebp
 ret

```

Based on the assembly code, fill in the blanks below in its corresponding C source code. (Note: you may only use symbolic variables `a`, `b`, and `result` from the source code in your expressions below — **do not** use register names.)

```

int frog(int a, int b)
{
 int result;

 switch (_____)
 {
 case _____:
 case _____:
 result = _____;
 break;

 case _____:
 result = _____;
 }
}

```

```

 break;

 case ____:
 _____;

 case 7:
 result = _____;
 break;

 default:
 result = _____;
}

return result;
}

```

### Problem 7:

This problem tests your understanding of the IA32 calling convention. Consider the following C code and corresponding assembly. Fill in the missing instructions (one instruction per a blank line).

```

int global;

int bear(int i, int j, int k)
{
 for (; i < j; i++)
 {
 global += k * i;
 }
 return global;
}

```

```

bear:
 pushl %ebp
 movl %esp, %ebp

 movl 8(%ebp), %edx
 movl 12(%ebp), %ebx
 movl 16(%ebp), %esi
 cmpl %ebx, %edx
 jge .L7
 movl global, %ecx
.L5:
 movl %esi, %eax
 imull %edx, %eax
 leal (%ecx,%eax), %ecx
 incl %edx
 cmpl %ebx, %edx
 jl .L5
 movl %ecx, global
.L7:

 popl %ebp
 ret

```



---

### Problem 8:

Consider the following x86-64 assembly function:

```
loop:
 # on entry: a in %rdi, n in %esi
 movl $0, %r8d
 movl $0, %ecx
 testl %esi, %esi
 jle .L3
.L6:
 movl (%rdi,%rcx,4), %edx
 leal 3(%rdx), %eax
 testl %edx, %edx
 cmovns %edx, %eax
 sarl $2, %eax
 addl %eax, %r8d
 addq $1, %rcx
 cmpl %ecx, %esi
 jg .L6
.L3:
 movl %r8d, %eax
 ret
```

Fill in the blanks of the corresponding C code.

- You may only use the C variable names `n`, `a`, `i` and `sum`, not register names.
- Use array notation in showing accesses or updates to elements of `a`.

```
int loop(int a[], int n)
{
 int i, sum;

 sum = ____;

 for (i = ____; ____; ____) {

 sum += ____;

 }

 return ____;
}
```

### Problem 9:

Consider the following x86-64 assembly function:

---

```

loopy:
 # a in %rdi, n in %esi
 movl $0, %ecx
 movl $0, %edx
 testl %esi, %esi
 jle .L3
.L6:
 movslq %edx, %rax
 movl (%rdi,%rax,4), %eax
 cmpl %eax, %ecx
 cmovl %eax, %ecx
 addl $1, %edx
 cmpl %ecx, %esi
 jg .L6
.L3:
 movl %ecx, %eax
 ret

```

Fill in the blanks of the corresponding C code.

- You may only use the C variable names **n**, **a**, **i** and **x**, not register names.
- Use array notation in showing accesses or updates to elements of **a**.

```

int loopy(int a[], int n)
{
 int i;
 int x = ____;

 for (i = ____; ____; ____){

 if (____)

 x = ____;
 }
 return x;
}

```

### Problem 10:

Answer the following questions about x86 stack convention (in 32-bit mode):

- How does the `call` instruction modify the stack?
- How does the `leave` instruction modify the stack?

- 
- C. How does the `ret` instruction modify the stack?
  - D. How are arguments passed to a function?
  - E. How are return values passed back to a calling function?

## PROBLEM SET 03: The Memory Hierarchy

Exercise in the textbook **Computer Systems: A Programmer's Perspective**: 6.23(hw), 6.24(hw), 6.2(Practice Problem), 6.3(Practice Problem), 6.4(Practice Problem), 6.5(Practice Problem),...

### Problem 1: Multiple choice questions

1. Which types of locality are leveraged by virtual memory?
  - (a) Spatial locality
  - (b) Temporal locality
  - (c) Prime locality
  - (d) Both (a) and (b)
  - (e) Both (b) and (c)
2. Which one of the following statements about cache memories is true:
  - (a) Larger caches are more susceptible to capacity misses than smaller caches.
  - (b) Caches with lower associativity are more susceptible to conflict misses than those with higher associativity.
  - (c) Caches with higher associativity are more susceptible to cold misses than those with lower associativity.
  - (d) None of the above
3. What is the maximum number of page faults per second that can be serviced in a system with a disk that has the following characteristics: 10,000 RPM rotation speed (6ms per full revolution), average seek time of 7ms, 1000 sectors per track. (Assume that all in-memory pages that get replaced are clean.)

- 
- (a) 50
  - (b) 100
  - (c) 77
  - (d) Not enough information to determine the answer
4. Consider an SRAM-based cache for a DRAM-based main memory.  
Neglect the possibility of other caches or levels of the memory hierarchy below main memory.  
If a cache is improved, increasing the typical hit rate from 98% to 99%, which one of the following would best characterize the likely decrease in typical access time?
- (a) 0%
  - (b) 1%
  - (c) 10%
  - (d) 100%
5. Two-dimensional arrays are stored in order, to help with cache performance.
- (a) column-major
  - (b) row-major
  - (c) diagonal-major
  - (d) Art-major

## PROBLEM SET 04: Exceptional Control Flow

Exercise in the textbook **Computer Systems: A Programmer's Perspective**:

Hw: 8.9, 8.10, 8.11, 8.12, 8.13, 8.15, 8.16, 8.18, 8.21, 8.19, 8.24,...

Practice Problems: 8.2, 8.3, 8.4,...

### Problem 1: Multiple choice questions

1. Assuming no errors, which one of the following functions returns exactly once?
  - (a) `fork()`
  - (b) `execve()`
  - (c) `exit()`
  - (d) `longjmp()`
  - (e) `waitpid()`
2. When can short counts occur?
  - (a) When an EOF is encountered during a read
  - (b) When a short int is used as a counter
  - (c) When reading or writing to disk files
  - (d) When the kernel runs out of kernel memory

- 
3. A program blocks SIGCHLD and SIGUSR1. It is then sent a SIGCHLD, a SIGUSR1, and another SIGCHLD, in that order. What signals does the program receive after it unblocks both of those signals (you may assume the program does not receive any more signals after)?
- (a) None, since the signals were blocked they are all discarded.
  - (b) Just a single SIGCHLD, since all subsequent signals are discarded.
  - (c) Just a single SIGCHLD and a single SIGUSR1, since the extra SIGCHLD is discarded.
  - (d) All 3 signals, since no signals are discarded.
4. Which of the following events does not generate a signal?
- (a) Division by zero
  - (b) A new connection arrives on a listening socket
  - (c) A write is attempted on a disconnected socket
  - (d) NULL is dereferenced
  - (e) A process whose parent has already terminated exits
5. Assuming no errors, which one of the following statements about `fork` is true?
- (a) Called once, returns once.
  - (b) Called once, returns twice.
  - (c) Called once, returns never.
  - (d) Called twice, returns once.
  - (e) None of the above.

## Problem 2:

Suppose that the following C program is run on a single processor machine.

```
#include <unistd.h>
#include <stdio.h>

int cnt = 0;

int main(void)
{
 if (fork() == 0) {
 cnt++;
 fork();
 cnt++;
 }
 cnt++;
 printf("%d", cnt);
 return 0;
}
```

List all possible outputs of this program.

---

### Problem 3:

This problem tests your understanding of Unix process control. Consider the following C program. (For space reasons, we are not checking error return codes, so assume that all functions return normally.)

```
int main()
{
 int status;
 int counter = 1;

 if (fork() == 0) {
 counter++;
 printf("%d", counter);
 }
 else {
 if (fork() == 0) {
 printf("9");
 counter--;
 printf("%d", counter);
 exit(0);
 }
 else {
 if (wait(&status) > 0) {
 printf("6");
 }
 }
 }

 printf("8");
 exit(0);
}
```

For each of the following strings, circle whether (Y) or not (N) this string is a possible output of the program.

| Output    | Y | N |
|-----------|---|---|
| A. 298068 |   |   |
| B. 291688 |   |   |
| C. 920688 |   |   |
| D. 268908 |   |   |
| E. 906828 |   |   |

---

#### Problem 4:

A. Consider the following C program. Assume the program executes to completion and that `fork`, `waitpid`, and `printf` always succeed.

```
int main() {
 pid_t pid;
 int sum = 0;

 if ((pid = fork()) == 0)
 sum += 10;
 else {
 waitpid(pid, NULL, 0);
 sum -= 5;
 }
 sum += 20;

 if (pid > 0)
 printf("Parent: sum=%d\n", sum);
 else
 printf("Child: sum=%d\n", sum);
 return 0;
}
```

Show the output of this program:

Child: sum=\_\_\_\_\_

Parent: sum=\_\_\_\_\_

B. Now consider the same program as in Part A, but with the call to `waitpid` removed. Assume the program executes to completion and that `printf` always succeeds. **Make no assumptions about the results of the other function calls.**

List all of the possible outputs of such a program. Each blank box holds the complete output from one execution of the program. Some blank boxes may be left unused.

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |

C. Consider the C program below. Assume the program runs to completion and that all functions return normally.

```
int main ()
{
 if (fork() == 0) {
```

---

```

 if (fork() == 0) {
 printf("9");
 exit(1);
 }
 else
 printf("5");
 }
 else {
 pid_t pid;
 if ((pid = wait(NULL)) > 0) {
 printf("3");
 }
 }
 printf("0");
 return 0;
}

```

List four possible outputs for this program:

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_
4. \_\_\_\_\_

## PROBLEM SET 05: Concurrent Programming with Threads

Exercise in the textbook **Computer Systems: A Programmer's Perspective**:

Hw: 12.17

Practice Problems: 12.6, 12.7, 12.9

**pthread\_create** - create a new thread

```

int pthread_create(pthread_t * thread,
 pthread_attr_t * attr,
 void * (*start_routine)(void *),
 void * arg);

```

**pthread\_self** - return identifier of current thread

```

pthread_t pthread_self(void);

```

**pthread\_exit** - terminate the calling thread

```

void pthread_exit(void *retval);

```

**pthread\_join** - wait for termination of another thread

```

int pthread_join(pthread_t th, void **thread_return);

```

**pthread\_detach** - put a running thread in the detached state



---

```
int pthread_detach(pthread_t th);
```

```
pthread_cancel - thread cancellation
int pthread_cancel(pthread_t thread);
```

Semaphores

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t * sem); /* P */
int sem_post(sem_t * sem); /* V */
```

### Problem 1:

This problem tests your understanding of race conditions in concurrent programs.

Consider a simple concurrent program with the following specification: The main thread creates two peer threads, passing each peer thread a unique integer *thread ID* (either 0 or 1), and then waits for each thread to terminate. Each peer thread prints its thread ID and then terminates.

Each of the following programs attempts to implement this specification. However, some are incorrect because they contain a race on the value of `myid` that makes it possible for one or more peer threads to print an incorrect thread ID. Except for the race, each program is otherwise correct.

You are to indicate whether or not each of the following programs contains such a race on the value of `myid`.

A. Does the following program contain a race on the value of `myid`?    **Yes**      **No**

```
void *foo(void *vargp) {
 int myid;
 myid = *((int *)vargp);
 Free(vargp);
 printf("Thread %d\n", myid);
}

int main() {
 pthread_t tid[2];
 int i, *ptr;

 for (i = 0; i < 2; i++) {
 ptr = Malloc(sizeof(int));
 *ptr = i;
 Pthread_create(&tid[i], 0, foo, ptr);
 }
 Pthread_join(tid[0], 0);
 Pthread_join(tid[1], 0);
}
```

B. Does the following program contain a race on the value of `myid`?    **Yes**      **No**

```
void *foo(void *vargp) {
```

---

```

 int id;
 id = *((int *)vargp);
 printf("Thread %d\n", id);
}

int main() {
 pthread_t tid[2];
 int i;

 for (i = 0; i < 2; i++)
 pthread_create(&tid[i], NULL, foo, &i);
 pthread_join(tid[0], NULL);
 pthread_join(tid[1], NULL);
}

```

C. Does the following program contain a race on the value of myid?    **Yes**    No

```

sem_t s; /* semaphore s */

void *foo(void *vargp) {
 int id;
 P(&s);
 id = *((int *)vargp);
 V(&s);
 printf("Thread %d\n", id);
}

int main() {
 pthread_t tid[2];
 int i;

 sem_init(&s, 0, 1); /* S=1 INITIALLY */

 for (i = 0; i < 2; i++) {
 Pthread_create(&tid[i], 0, foo, &i);
 }
 Pthread_join(tid[0], 0);
 Pthread_join(tid[1], 0);
}

```

## Problem 2:

Consider the following three threads and four semaphores:

```

/* Initialize x */
x = 1;

/* Initialize semaphores */

```

---

```
s1 =
s2 =
s3 =
s4 =

void thread1() {
 while (x != 360) {

 x = x * 2;

 }
 exit(0);
}

void thread2() {
 while (x != 360) {

 x = x * 3;

 }
 exit(0);
}

void thread3() {
 while (x != 360) {

 x = x * 5;

 }
 exit(0);
}
```

Provide initial values for the four semaphores and add P(), V() semaphore operations (using the four semaphores) in the code for `thread1`, `thread2` and `thread3` such that the process is guaranteed to terminate.

### Problem 3:

Consider three concurrently executing threads in the same process using two semaphores `s1` and `s2`. Assume `s1` has been initialized to 1, while `s2` has been initialized to 0.

What are the possible values of the global variable `x`, initialized to 0, after all three threads have terminated?

---

```

/* thread A */
P(&s2);
P(&s1);
x = x * 2;
V(&s1);

/* thread B */
P(&s1);
x = x * x;
V(&s1);

/* thread C */
P(&s1);
x = x + 3;
V(&s2);
V(&s1);

```

#### Problem 4:

This problem tests your understanding of race conditions in concurrent programs.

Consider a simple concurrent program with the following specification: The main thread creates two peer threads, passing each peer thread a unique integer **thread ID** (either 0 or 1), and then waits for each thread to terminate. Each peer thread prints its thread ID and then terminates.

Each of the following programs attempts to implement this specification. However, some are incorrect because they contain a race on the value of **myid** that makes it possible for one or more peer threads to print an incorrect thread ID. Except for the race, each program is otherwise correct.

You are to indicate whether or not each of the following programs contains such a race on the value of **myid**. You will be graded on each subproblem follows:

**A. Does the following program contain a race on the value of **myid**?    Yes    No**

```

void *foo(void *vargp) {
 int myid;
 myid = *((int *)vargp);
 Free(vargp);
 printf("Thread %d\n", myid);
}

int main() {
 pthread_t tid[2];
 int i, *ptr;

 for (i = 0; i < 2; i++) {
 ptr = Malloc(sizeof(int));
 *ptr = i;
 Pthread_create(&tid[i], 0, foo, ptr);
 }
}

```

---

```
 Pthread_join(tid[0], 0);
 Pthread_join(tid[1], 0);
}
```

**B. Does the following program contain a race on the value of myid?    Yes    No**

```
void *foo(void *vargp) {
 int id;
 id = *((int *)vargp);
 printf("Thread %d\n", id);
}

int main() {
 pthread_t tid[2];
 int i;

 for (i = 0; i < 2; i++)
 Pthread_create(&tid[i], NULL, foo, &i);
 Pthread_join(tid[0], NULL);
 Pthread_join(tid[1], NULL);
}
```

**C. Does the following program contain a race on the value of myid?    Yes    No**

```
void *foo(void *vargp) {
 int id;
 id = (int)vargp;
 printf("Thread %d\n", id);
}

int main() {
 pthread_t tid[2];
 int i;

 for (i = 0; i < 2; i++)
 Pthread_create(&tid[i], 0, foo, i);
 Pthread_join(tid[0], 0);
 Pthread_join(tid[1], 0);
}
```